

観測事後整理システム構想

技術補強版 ver0.1

作成日: 2026年6月4日

バージョン: ver0.1

実用できる最低ライン(観測写真のフォルダパスを受け取って、その写真についての各種情報を整理する)

○基本使用形態

使用手順

1. 観測時に独自作成の気象条件観測システムで気象条件を観測
1. after_kansoku.exeを実行
2. 観測データパスの取得※
3. 保存先を設定※
4. 実行
5. 保存先にデータベースが保存される(SQL形式)
6. 必要に応じてdb→xlsxに複製変換してエンドユーザーも閲覧できるようにします。(foolproofのために一方通行です)

after_kansoku.exeではMIN2, seeing_allframeの実行も行います。

※については、その内容をsetting.txtで設定できるようにします。

生データの構成

```
E:.\
├─observed_result
│   ├── environ_yy-mm-dd-tt.txt
│   └── weather_results.db
└─pictures
    ├── 001
    │   ├── 00001.tiff
    │   ├── 00002.tiff
    │   ├── 00001.txt
    │   └── 00002.txt
    └── 002
        ├── 00001.txt
        ├── 00001.tiff
        └── 00002.tiff
```

この形式を想定して作製します。

○データベースの構造

photos: 写真準拠のデータを写真に紐づけて保管

(観測は、各写真の撮影データが必要なのでtiff撮影でおこなう)

- camera_stat (撮影時に生成されるtxtfile)
- 写真のパス
- 撮影時刻 (time stomp: 秒精度)
- 解析結果 (MIN2, seeing_allframe)
 - **【技術定義】** データベースのカラムとして、太陽の半径(sun_radius)、中心座標(center_x, center_y)、シーイング値(img_seeing)などを数値 (REAL型) として具体的に定義・記録します。

verniers: 気象条件のデータを保管

(観測写真と、fps, タイミングが微妙に異なるため。また、気象条件のみに焦点を当てる場合もあるため)

- 観測時刻 (time stomp: ミリ秒精度)
 - **【技術定義】** `pd.date_range(freq='7.8125ms')`等を用いて、ミリ秒単位のズレのない精緻なタイムスタンプを自動生成し、写真との正確な紐付けを担保します。
- 各種数値 (湿度、気圧...etc)
 - **【技術定義】** Vernierセンサーからのデータ取得には公式のgodirectモジュールを使用します。また、128Hzの高頻度データを1件ずつ書き込むとPCがフリーズするため、1秒分 (128個) のデータを配列に溜め、`pandas.to_sql`を用いて「一括保存 (バルクインサート)」することで負荷を大幅に軽減します。

→写真と気象条件を組み合わせるときは、time stompでbetween検索する。

○コードの構造

```
E: .
├── basement
│   ├── sql_sample.py
│   ├── pandas_sample.py
│   ├── testDB.db
│   ├── 実装したいこと.md
│   └── 観測事後整理システム構想ver0.1.md
├── observation
│   ├── get_enviroment.py
│   └── west_solar.py
└── main
    ├── picture_analysis.py
    ├── setting.txt
    └── edit_database.py
```

main

- **setting.txt**
savepathやpicpath, ゆくゆくはoptionを保存しています。これらはデータの読み書きの際に使われます。
- **picture_analysis.py**
camera_statやMIN2, allframeの処理を行います。モジュール化します。
- **edit_database.py**
一番メインですヨ
全体の司令塔の役割になります。

observation

- **get_enviroment.py**
気象条件の観測をします。一時できなDBに書き込んで保存し、あとで吸い上げてもらって保存するつもりです。デバイスや時刻を保存します。
- **west_solar.py**
撮影時に、太陽の東西を検出するために使います。之については、多重露光をするものとどちらが良いかは要検討です。(afterだとピクセルが汚い)

basement

- **sql_sample.py**
pythonでsqlを用いた初歩的なデータベース操作です。デモなので実際の処理系とは異なる場合があります。

- **pandas_sample.py**

pythonでpandasを用いた初歩的なデータベース操作です。デモなので実際の処理系とは異なる場合があります。

- **testDB.db**

上記pandas_sample.pyで作られるdbfileです

- **実装したいこと.md**

追加したい機能や修正したいことを書きます。主にUI,UXについてです。

- **観測事後整理システム構想ver0.1.md**

本文ですが、ほぼREADMEに近いです。実装したいこと.md を参考にしながら構想を練ります。

〇たすく

- 画像解析ソフトの作成(MIN2, seeing_allframe内包でS Q Lを吐き出す)
- camera_statの展開ソフトの作成
- vernierのデータ収集ソフトの作成(観測機器、観測時間などを詳細に記録。高精度タイムスタンプ付与とバルクインサート機構を含む)
- db_to_xslsの複製変換ソフトの作成
- 太陽の東西検出ソフトの作成